

1. Introduction

Pour ce premier rendu axé sur les classes Java, les requêtes JDBC et l'interface console, j'ai pris en charge le développement de plusieurs classes réparties selon l'architecture de notre projet :

La logique du programme (Classes Java) :

- **Categorie et Couleur** : J'ai créé ces classes pour représenter les caractéristiques des pièces de LEGO en Java, en traduisant les entités correspondantes du schéma de la base de données (MCD). J'ai fait attention à respecter l'encapsulation en utilisant des variables privées et des méthodes d'accès (getters et setters).
- **EtatBoite** : J'ai développé cette classe pour gérer la logique de l'état des boîtes de la collection (permettant d'indiquer si une boîte est complète ou incomplète par exemple).

L'interface utilisateur (Console) :

- **GestionConfirmation** : L'application fonctionnant pour l'instant uniquement en ligne de commande avec des print dans la console. Elle permet d'afficher des messages et de demander une validation textuelle à l'utilisateur (comme taper "O" ou "N") avant d'exécuter une action, évitant ainsi les erreurs de manipulation.

Le lien avec la base de données (JDBC) : J'ai codé les classes qui s'occupent des requêtes SQL pour interroger et modifier les tables d'association complexes de la base de données:

- **ContenirPBD** : Gestion des requêtes pour les pièces contenues dans une boîte.
- **ContenirFBD** : Gestion des requêtes pour les figurines présentes dans les boîtes.
- **ContenirBBD** : Gestion des requêtes pour les boîtes incluses dans d'autres boîtes, comme les packs.

2. Difficultés rencontrées

La principale difficulté a été l'approfondissement de JDBC en connectant les requêtes à une base de données bien plus développée que ce qu'on a pu voir en TP avec des requêtes simples.

Au début, j'ai eu beaucoup de mal avec la gestion des clés étrangères dans les tables de liaison (`CONTENIRP`, `CONTENIRF`, `CONTENIRB`). Lier des listes d'objets Java avec ces tables provoquait souvent des erreurs de type `NullPointerException` ou des violations de contraintes en base de données. Pour m'en sortir, j'ai dû apprendre à tester mes requêtes SQL directement sur la base de données avant de les intégrer dans mon code Java. J'ai aussi passé beaucoup de temps à faire des affichages de vérification dans la console pour comprendre précisément d'où venaient les bugs et corriger la logique de mes requêtes.

3. Compétences validées et apprentissages

Ce travail sur le premier jet m'a permis de mettre en pratique et de valider plusieurs compétences de première année :

- **Développer et concevoir simplement (AC11.01 et AC11.02)** : J'ai appris à lire un modèle conceptuel de données (MCD) et à l'implémenter de façon logique sous forme de classes orientées objet en Java.
- **Faire des essais et évaluer les résultats (AC11.03)** : J'ai dû mettre en place une démarche de test rigoureuse pour mes fonctions JDBC. En utilisant la console pour afficher les résultats de mes requêtes, j'ai vérifié que les données extraites ou insérées correspondaient exactement aux spécifications attendues.
- **Qualité de développement (R2.03)** : J'ai approfondi l'utilisation de la JavaDoc pour documenter proprement mes classes afin qu'elles soient compréhensibles par le reste du groupe, et j'ai commencé à intégrer la gestion des exceptions pour rendre le menu console plus robuste face aux erreurs de saisie.

4. Conclusion

Ce premier rendu m'a permis d'améliorer mon niveau en Java et de mieux comprendre son fonctionnement. Jusqu'ici, nous avions l'habitude de coder des petits programmes simples et isolés. Avec ce projet, j'ai découvert comment on lie concrètement du code Java avec une véritable base de données notamment avec la partie JDBC.

Réussir à faire fonctionner cette interface console avec nos classes et nos requêtes JDBC me donne une base solide et beaucoup plus d'assurance pour aborder la suite du développement de l'application et cela me donne un exemple concret de ce qui est attendu dans le domaine professionnel.